

Install Required Libraries

```
pip install tensorflow numpy matplotlib
```

Import Libraries

```
import numpy as np
import tensorflow as tf
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import LSTM, Dense, SimpleRNN
```

Task 1: Build an LSTM Model for Daily Step Prediction

```
import numpy as np
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import LSTM, Dense
```

```
# Daily step counts
```

```
steps = np.array([
    4000,
    4200,
    4300,
    4100,
    4500,
    4700,
    4800,
    5000
], dtype=float)
```

```
# Normalize
```

```
steps = steps / 5000
```

```
# Create sequences
```

```
X = []
```

```
y = []
```

```
for i in range(len(steps)-3):
```

```
    X.append(steps[i:i+3])
```

```
    y.append(steps[i+3])
```

```
X = np.array(X)
```

```
y = np.array(y)
```

```

# Reshape for LSTM
X = X.reshape((X.shape[0], X.shape[1], 1))

# Build Model
model = Sequential([
    LSTM(32, input_shape=(3,1)),
    Dense(1)
])

model.compile(
    optimizer="adam",
    loss="mse"
)

# Train
model.fit(
    X,
    y,
    epochs=100,
    verbose=1
)

# Predict
prediction = model.predict(X[-1].reshape(1,3,1))

print("Predicted Next Step Count:", prediction[0][0]*5000)

```

Explanation

- LSTM learns the pattern in previous daily steps.
- The model predicts the next day's step count based on the last three days.

Task 2: Demonstrate Vanishing Gradient in a Basic RNN

```

import tensorflow as tf
import numpy as np

# Dummy sequence
X = np.random.randint(0,2,(20,10,1)).astype(np.float32)
y = np.random.randint(0,2,(20,1)).astype(np.float32)

model = tf.keras.Sequential([
    tf.keras.layers.SimpleRNN(

```

```

        8,
        input_shape=(10,1)
    ),
    tf.keras.layers.Dense(1, activation="sigmoid")
])

```

```

optimizer = tf.keras.optimizers.Adam()
loss_fn = tf.keras.losses.BinaryCrossentropy()

```

```

for epoch in range(5):

```

```

    with tf.GradientTape() as tape:

```

```

        pred = model(X)

```

```

        loss = loss_fn(y,pred)

```

```

        gradients = tape.gradient(
            loss,
            model.trainable_variables
        )

```

```

        optimizer.apply_gradients(
            zip(
                gradients,
                model.trainable_variables
            )
        )

```

```

        print(f"\nEpoch {epoch+1}")

```

```

    for g in gradients:

```

```

        print(tf.reduce_mean(tf.abs(g)).numpy())

```

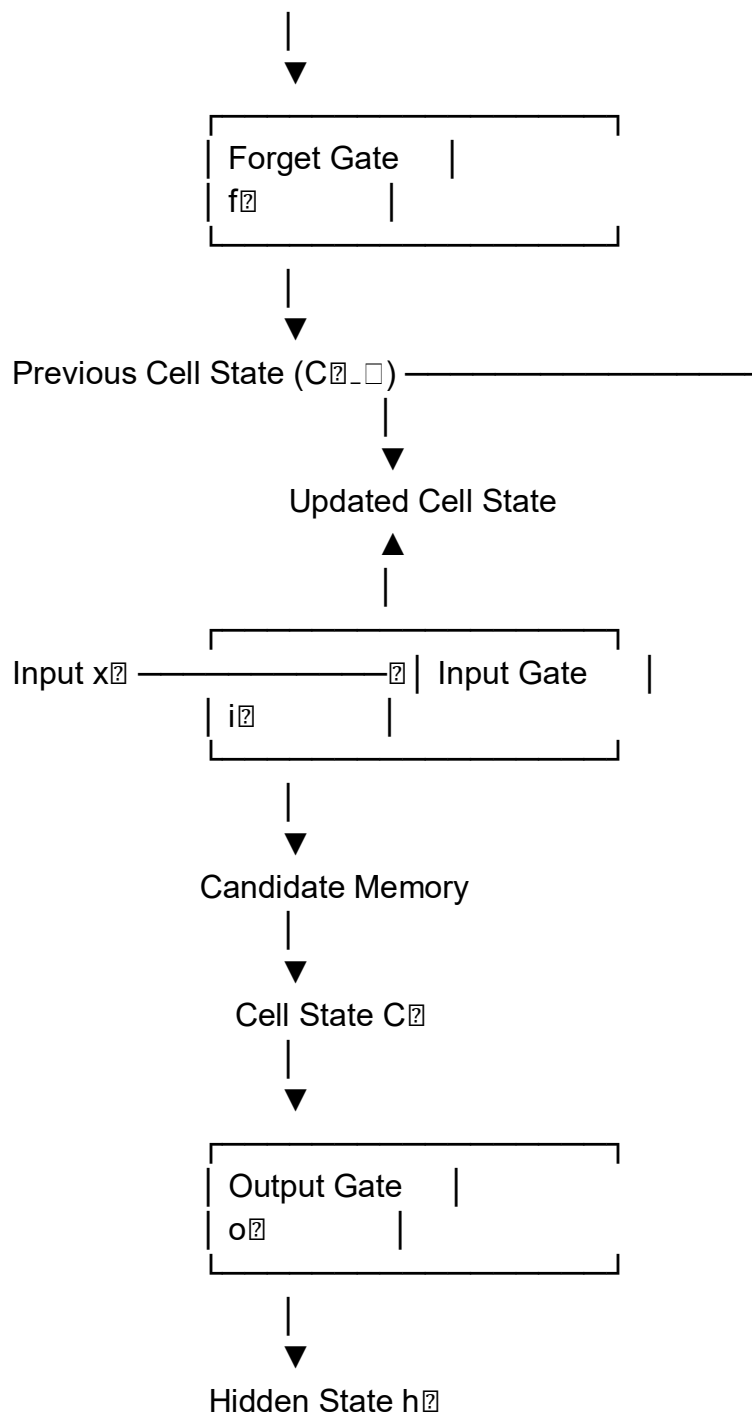
Observation

The gradients may become very small in deeper or longer sequences, illustrating the vanishing gradient effect that can hinder learning.

Task 3: Architecture of a Single LSTM Cell

Previous Hidden State (h_{t-1})

|



Gate Functions

Gate	Function
Forget Gate	Decides what information from the previous cell state should be discarded.
Input Gate	Determines what new information should be stored in the cell state.
Cell State	Carries long-term information through the sequence.

Gate	Function
Output Gate	Chooses which information from the cell state becomes the hidden state/output.

Task 4: LSTM Information Flow (Zomato Example)

Input Sequence

[pizza, burger, pasta, pizza]

Assume the cell has already processed **pizza**.

Time Step 1

Current Input:

burger

Gate	Value	Action
Forget Gate	1	Keep previous preference for pizza.
Input Gate	1	Store the new preference for burger.
Output Gate	1	Output both pizza and burger preferences.

Cell State:

[pizza, burger]

Time Step 2

Current Input:

pasta

Gate	Value	Action
Forget Gate	0	Forget the older pizza preference.
Input Gate	1	Add pasta as a new preference.
Output Gate	1	Output burger and pasta preferences.

Updated Cell State:

[burger, pasta]

Interpretation

The LSTM selectively keeps useful long-term information and removes less relevant information, allowing it to adapt to changing user preferences over time.

Task 5: Unrolling an LSTM Through Time

Explanation (3–4 lines):

Unrolling an LSTM through time means representing the same LSTM cell repeatedly across each time step of a sequence while sharing the same weights. For example, in a music playlist app, the LSTM processes each song a user listens to one after another, carrying forward its memory of previous songs. This enables the model to use listening history to predict the next song the user is most likely to play.

Final Summary

Task Implementation	Key Learning
1 Built an LSTM model to predict the next daily step count from previous step sequences.	LSTMs capture temporal patterns and use past information for sequence prediction.
2 Trained a simple RNN and printed gradients after each epoch.	Demonstrates how gradients can become very small, illustrating the vanishing gradient problem.
3 Drew and labeled an LSTM cell architecture.	Shows the roles of the forget gate, input gate, cell state, and output gate in managing information flow.
4 Walked through two LSTM time steps using a Zomato food-order example.	Demonstrates how LSTMs decide what information to keep, forget, and output over time.
5 Explained unrolling an LSTM through time with a music playlist example.	Unrolling illustrates how the same LSTM cell processes sequential inputs while maintaining shared parameters and memory across time steps.